University of Information Technology – VNU-HCM
Faculty of Computer Engineering

CSBU102 – Computer System

LAB 02

Accessing Main Memory: Words, Arrays, Bytes and Strings

Full name	Trần Hữu Đạt	**Student ID**	26560011
Class	ATBC2026	**Practice date**	11/11/2026
Self-assessed score	100	**GitHub link**	https://github.com/MonstrousGreyEagle/CSBU102.R11.KHBC

1. LAB OVERVIEW

Purpose. In LAB 1 every value lived in one of the 32 registers. Real programs use far more data than that, such as a list of scores or a line of text, and keep it in main memory (RAM); a processor can calculate only on registers. In this session you will learn how a program finds data in memory, brings it into a register and writes a result back, for whole words (arrays) and for single bytes (characters). Every new idea is explained first and then practised.

> **Situation: a lecturer's score sheet**
> A lecturer keeps the scores of a class and the students' names in a small program. Even five scores and one name are too much to keep in registers, so the program stores them in main memory and moves one value at a time into a register to read, change or add it. All names and scores are fictitious.

After this session, you can:

- explain address, word and the alignment rule, and declare data with `.word`, `.byte` and `.asciiz`;
- read and write words with `lw` and `sw` (base register + offset) and compute the address of array element i as base + 4 × i, also when i is held in a register;
- read and write bytes with `lb`, `lbu` and `sb`, and explain sign extension and byte order from your own observations;
- change data in memory: swap two elements, update an element, edit characters of a string;
- predict, observe and explain what a short program does with memory, including what happens when it goes wrong.

Table 1. Structure of the session

Part	Content	Score
0	Getting started: the memory panes of MARS	–
1	Experiment 1 – Words and addresses	30
2	Experiment 2 – Arrays: reading and writing elements	30
3	Experiment 3 – Bytes, characters and strings	25
4	Wrap-up and submission	15
	Total	**100**
Ext	Extension – A 3 × 3 board in memory (optional)	+10

Working conventions

• A yellow cell is where you write an answer or a value (your text appears in red). A dashed frame is where you paste a screenshot. Listings are in grey frames; a line that starts with `# >>>` marks a place where you write code.

• Questions are labelled *Understand*, *Apply*, *Analyze* or *Evaluate*. Many activities follow PREDICT → OBSERVE → EXPLAIN: write down what you expect, run the program and record what happens, then explain any difference. A wrong prediction is never penalised; the explanation earns the marks.

• Save every program you submit as `Lab02_<StudentID>_<Part>.asm`, for example `Lab02_22520001_TN2.asm` (parts TN1 to TN3 and EXT), and start it with a comment block with your name, student ID and a one-line purpose.

PART 0. GETTING STARTED: THE MEMORY PANES OF MARS

You already know how to assemble, run and step through a program. This time you also use the Data Segment, the MARS pane that shows main memory (Table 2).

Table 2. Skills checklist

☐	Skill	Hint
x	Check the settings	Settings: tick *Permit extended (pseudo) instructions* and *Initialize Program Counter to global 'main'*.
x	Read the Data Segment	Each row starts with an address; the columns *Value (+0)*, *Value (+4)*, … show the words that follow. Tick *Hexadecimal Values* / *Addresses* or *ASCII* to change the display.
x	Find the address of a label	Settings → *Show Labels Window (symbol table)*.
x	Set a breakpoint	Tick the *Bkpt* box of an instruction in the Text Segment; Go stops just before it.

Step 0 – Quick check. Type Listing 0, assemble and run it (it prints one number read from memory). Then find the label `x` in the Labels window and the number 7 in the Data Segment.

Listing 0

```
# Listing 0 - quick check
.data
x:    .word 7
msg:  .asciiz "Value read from memory: "
nl:   .asciiz "\n"

.text
.globl main
main:
    li   $v0, 4
    la   $a0, msg
    syscall
```

```
    la   $t0, x
    lw   $a0, 0($t0)
    li   $v0, 1
    syscall
    li   $v0, 4
    la   $a0, nl
    syscall
    li   $v0, 10
    syscall
```

Text in the Run I/O pane:

```
Value read from memory: 7
```

EXPERIMENT 1. WORDS AND ADDRESSES (30 points)

Situation

Before the program can add two scores it must find them. A processor sees not a list of named scores but a very long row of numbered storage cells. This experiment shows how a program names a cell, brings its contents into a register and writes a result back.

Background: main memory, addresses and words

Main memory is a long row of **bytes** (8 bits). Every byte has its own number, its **address**. A MIPS register holds one **word** of 4 bytes, which occupies four consecutive addresses; the address of a word is the address of its first byte and must be a **multiple of 4** (the *alignment rule*). In MARS, declared data begins at `0x10010000` (Figure 1).

The processor calculates only with registers, so a program **loads** a value into a register and **stores** a result back: `lw $t0, 8($s0)` copies the word at address `$s0 + 8` into `$t0`, and `sw $t0, 8($s0)` copies `$t0` into that word. The constant 8 is the **offset** (in bytes), `$s0` is the **base register**, and base + offset is the *effective address*. A label names an address, and `la $s0, scores` copies the address of `scores` into `$s0`. Table 3 lists the directives that place data in memory.

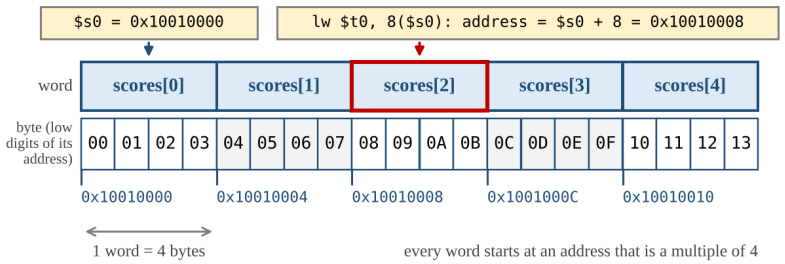


Figure 1. An array of five words in memory and the effective address of lw $t0, 8($s0)

Table 3. Data declarations

Directive	Example	What is placed in memory
`.word`	`a: .word 40, 55, 70`	Consecutive 32-bit words (an array); the label is the address of the first one.
`.byte`	`b: .byte 65`	One byte with the value 65.
`.asciiz`	`s: .asciiz "Hi"`	One byte per character, then one extra byte of value 0 that marks the end of the string.

Step 1.1 – Variables in memory (10 points)

In LAB 1, `a = b + c` was a single `add` on registers. Now the variables live in memory. Type Listing 1, assemble it, PREDICT the values in Table 4, then run it (decimal display) and OBSERVE. Also read the addresses in Table 5 (`pc` and `$sp` from the Registers pane right after assembling).

Listing 1

```
# Listing 1 - varA = varB + varC, with all three variables in memory
.data
varB: .word 25
varC: .word 17
varA: .word 0

.text
.globl main
main:
    la   $t0, varB         # $t0 = address of varB
    lw   $t1, 0($t0)       # $t1 = varB
    lw   $t2, 4($t0)       # $t2 = varC
    add  $t3, $t1, $t2     # $t3 = varB + varC
    sw   $t3, 8($t0)       # varA = $t3
    li   $v0, 10
    syscall
```

Table 4. Predict and observe Listing 1

Item	PREDICT (decimal)	OBSERVE (decimal)
`$t3`	42	42
Word at label `varA` in the Data Segment	42	42

Table 5. Addresses in the program

Item	Address (hex)
`pc` before the program runs	0x00400000
Label `varB` / `varC` / `varA`	0x10010000/0x10010004/0x10010008
`$sp` before the program runs	0x7fffeffc

Question 1.1 (Understand) Why do the addresses of `varB`, `varC` and `varA` differ by that number of bytes? The instructions, the declared data and the stack are kept in three different regions: which address belongs to which region, and which region has the highest addresses?

varA, varB and varC ‘s addresses differ by an increment of 0x4 because they are defined as word, which is 4 bytes long.
Instructions belongs to the .text segment, data belongs to the .data section and the stack belongs to the stack region. Among the three, the address of stack is the highest/

Step 1.2 – Base plus offset (10 points)

Type Listing 2 (an array of five scores). For each instruction in Table 6, work out the effective address and PREDICT the value that ends up in the destination register; run the program and record the observed value. Row (4) is an `addi`: its result is an address (hexadecimal).

Listing 2

```
# Listing 2 - addresses and offsets
.data
scores: .word 40, 55, 70, 85, 100

.text
.globl main
main:
    la   $s0, scores       # $s0 = address of scores[0]
    lw   $t0, 0($s0)       # (1)
```

```
    lw   $t1, 4($s0)      # (2)
    lw   $t2, 16($s0)     # (3)
    addi $s1, $s0, 8      # (4) $s1 = $s0 + 8
    lw   $t3, 0($s1)      # (5)
    lw   $t4, -4($s1)     # (6)
    li   $v0, 10
    syscall
```

Table 6. Effective addresses and loaded values

#	Instruction	Effective address (hex)	PREDICT value	OBSERVE value
(1)	`lw $t0, 0($s0)`	0x00400008	40	40
(2)	`lw $t1, 4($s0)`	0x0040000c	55	55
(3)	`lw $t2, 16($s0)`	0x00400010	100	100
(4)	`addi $s1, $s0, 8`	–	0x10010008	0x10010008
(5)	`lw $t3, 0($s1)`	0x00400018	70	70
(6)	`lw $t4, -4($s1)`	0x0040001c	55	55

Question 1.2 (Apply) Write the formula for the address of element `i` of an array of words in terms of the address of element 0, and use it for `scores[3]`. Why does 4 appear in the formula, and not 1?

let $s0=i; $s1=&scores

shl $t0, $s0, 2
add $t0, $t0, $s1
lw $t0, 0($t0) #load scores[i] to $t0

'4' appears in the code in "shl $t0, $s0, 2" because each element of the array is a word, which is 4 bytes in length thus require an increment of 4 in the address for each adjacent element

Step 1.3 – Break it on purpose (10 points)

For each change in Table 7, PREDICT what will happen, change instruction (2) of Listing 2 (one change at a time), run the program and OBSERVE. In (a), also note the message that MARS shows. Restore the program afterwards.

Table 7. Controlled experiments

Change	PREDICT	OBSERVE
(a) `lw $t1, 2($s0)`	error	Go: execution terminated with errors.
(b) `lw $t1, 20($s0)`	0	0

Question 1.3 (Analyze) Explain (a) and (b). Which one does the processor detect, and which does it not? What does this say about who is responsible for keeping an access inside an array?

the process return an error on (a) because 4 is not the divisor of 2, thus the lw (load word) instruction fail, because the address does not align with the word memory block (memories are divided into 0x4 bytes block in MIPS).

EXPERIMENT 2. ARRAYS: READING AND WRITING ELEMENTS (30 points)

Situation

The lecturer finds a mistake in the score sheet. Two scores must be exchanged, one score must be corrected, and the total of the five scores must be recalculated. All of this happens in memory, element by element.

> **Background: the address of element i**
> The elements of an array are words stored one after another, so element `i` is at **base + 4 × i** (Experiment 1). When the index is a constant, you write 4 × i yourself as the offset of `lw` or `sw`. When the index is held in a register, the program must compute the address: `sll $t1, $s1, 2` shifts `$s1` left by two bits, which multiplies it by 4, and `add $t1, $t1, $s0` adds the base address. Then `lw` or `sw` uses `0($t1)`.

Step 2.1 – Swap two scores (10 points)

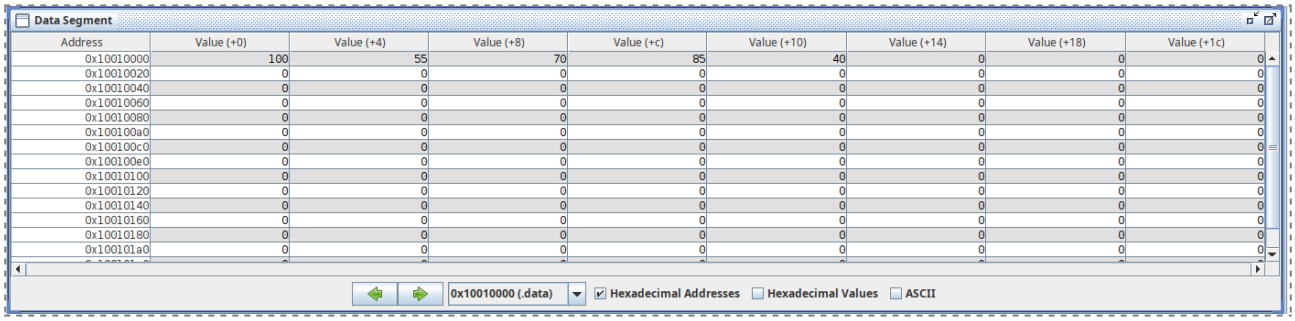
Complete Listing 3 so that it exchanges `scores[0]` and `scores[4]` in memory, using only `lw` and `sw`. After the run, the Data Segment must show 100, 55, 70, 85, 40 (decimal).

Listing 3

```
# Listing 3 - swap two scores in memory
.data
scores: .word 40, 55, 70, 85, 100

.text
.globl main
main:
    la   $s0, scores      # $s0 = address of scores[0]
    # >>> load scores[0] into $t0
    # >>> load scores[4] into $t1
    # >>> store $t1 into scores[0]
    # >>> store $t0 into scores[4]
    li   $v0, 10
    syscall
```

Screenshot of the Data Segment after the run:



Question 2.1 (Analyze) Why did you need two registers? Could one temporary register be enough? Explain.

In MIPS, it is impossible to directly manipulate the data saved in the memory. Thus, we must use two register to represent the two block of data used in the operation.

Step 2.2 – The index in a register (10 points)

Listing 4 computes the address of `scores[i]` from the index `i` in `$s1` and prints the element. Type it and run it with i = 3 to see how it works. Then, for each variant in Table 8, PREDICT the result (for (a) to (c): `$t1` in hexadecimal and `$t2`; for (d): what happens), change the program, run it and OBSERVE. Restore the original program after each variant.

Listing 4

```
# Listing 4 - the index of the element is held in a register
# Register plan:  $s0 = address of scores[0]   $s1 = i
.data
scores: .word 40, 55, 70, 85, 100
msg:    .asciiz "scores[i] = "
nl:     .asciiz "\n"

.text
.globl main
main:
    la   $s0, scores      # $s0 = address of scores[0]
```

```
    addi $s1, $zero, 3     # i = 3
    sll  $t1, $s1, 2       # $t1 = 4 * i
    add  $t1, $t1, $s0     # $t1 = address of scores[i]
    lw   $t2, 0($t1)       # $t2 = scores[i]
    li   $v0, 4
    la   $a0, msg
    syscall
    li   $v0, 1
    add  $a0, $t2, $zero
    syscall
    li   $v0, 4
    la   $a0, nl
    syscall
    li   $v0, 10
    syscall
```

Table 8. Variants of Listing 4

Variant	PREDICT	OBSERVE
(a) `i = 0`	40	40
(b) `i = 2`	70	70
(c) `i = 4`	100	100
(d) `i = 3` and `sll $t1, $s1, 1`	error	Go: execution terminated with errors.

Question 2.2 (Analyze) Why does the shift amount have to be 2? Explain what happened in (d) and which part of the computer detected the problem.

The process return an error on (d) because 4 is not the divisor of 2 (shift left 1), thus the lw (load word) instruction fail, because the address does not align with the word memory block (memories are divided into 0x4 bytes block in MIPS).
The shift ammount must be 2 because shl means shift the bits to the left, or multiply the value by a factor of 2 to the power of imm, and 4 (or the size of each element of the array) is 2 to the power of 2.

Step 2.3 – Your own data (10 points)

Let D be the last digit of your student ID. In Table 9, find the index k and the new score V. Write a program, starting from Listing 4, that (1) puts k in `$s1` and V in a register and stores V into `scores[k]` with `sw`, using the address computation of Step 2.2; (2) adds up the five scores in `$s3`, loading each one with `lw` and a constant offset; (3) prints `Total:` followed by the total and a new line. First run it without step (1): the total of the original scores is 350. Then PREDICT the total for your own data by hand, run the program and complete Table 10.

Table 9. Personal data (D = last digit of the student ID)

D	k	V	D	k	V
0	0	15	5	0	62
1	1	28	6	1	77
2	2	33	7	2	84
3	3	46	8	3	91
4	4	59	9	4	98

Table 10. Results for your own data

My D	Result	Total	scores[k] in the Data Segment
1	**PREDICT**	323	28
1	**OBSERVE**	323	28

EXPERIMENT 3. BYTES, CHARACTERS AND STRINGS (25 points)

Situation

The score sheet also contains students' names. Text is stored one character per byte, so the program must read and write single bytes, not only whole words.

Background: characters, bytes and byte order

A character is stored as a small number, its **ASCII code** (Table 18): `A` is 65 (`0x41`), `a` is 97 (`0x61`), the digit `0` is 48. A string declared with `.asciiz` is a sequence of codes followed by a byte of value 0. Three instructions work on single bytes (Table 11); a byte address can be any number.

A word occupies four consecutive bytes, so a computer must decide which byte of the number goes to the lowest address. In **little-endian** order the least significant byte comes first; in **big-endian** order the most significant byte comes first. Both are used in real computers, and the lecture slides show both. Step 3.2 shows which one MARS uses.

Table 11. Instructions that move single bytes

Instruction	Example	Meaning
`lb`	`lb $t0, 1($s0)`	Load the byte at address `$s0 + 1`; the upper 24 bits are filled with copies of the byte's top bit (sign extension).
`lbu`	`lbu $t0, 1($s0)`	Load the byte at address `$s0 + 1`; the upper 24 bits are filled with zeros.
`sb`	`sb $t0, 1($s0)`	Store the lowest 8 bits of `$t0` in the byte at address `$s0 + 1`.

Step 3.1 – The bytes of a string (6 points)

Type Listing 5 and assemble it; tick *ASCII* in the Data Segment. Using Table 18, PREDICT the hexadecimal code of each byte of `"Hi!"`; then run and complete Table 12 and the answer below it.

Listing 5

```
# Listing 5 - the bytes of a string
.data
word1: .asciiz "Hi!"

.text
.globl main
main:
    la   $s0, word1       # $s0 = address of the first byte
    lb   $t0, 0($s0)
    lb   $t1, 1($s0)
    lb   $t2, 2($s0)
    lb   $t3, 3($s0)
    li   $v0, 11          # service 11: print one character
    add  $a0, $t1, $zero  # the byte that was loaded into $t1
    syscall
    li   $v0, 10
    syscall
```

Table 12. The bytes of word1

Offset	Address (hex)	PREDICT byte (hex)	OBSERVE byte (hex)	Character
0	0x10010000	0x48	0x48	H
1	0x10010001	0x69	0x69	i
2	0x10010002	0x21	0x21	!
3	0x10010003	0x00	0x00	NULL

PREDICT – character printed in the Run I/O pane:

i

Question 3.1 (Understand) How many bytes does `"Hi!"` occupy, and why one more than the number of visible characters? What is the address of the last byte?

the string occupies 4 bytes, which is one more than its visible character. This happen because strings are NULL-terminated at the end, or rather, there exist a NULL “\x00” byte at the end of each string to mark the end of string.

Step 3.2 – Which byte comes first? (6 points)

Listing 6 stores the word `0x41424344` and reads its four bytes one by one. Decide which byte order you expect MARS to use and PREDICT the four bytes; then run it and complete Table 13.

Listing 6

```
# Listing 6 - one word, four bytes
.data
w: .word 0x41424344

.text
.globl main
main:
    la   $s0, w
    lb   $t0, 0($s0)      # byte at offset 0
    lb   $t1, 1($s0)      # byte at offset 1
    lb   $t2, 2($s0)      # byte at offset 2
    lb   $t3, 3($s0)      # byte at offset 3
    lw   $t4, 0($s0)      # the whole word
    li   $v0, 10
    syscall
```

Table 13. Reading the bytes of one word

Item	PREDICT (hex)	OBSERVE (hex)
Byte at offset 0 (`$t0`)	0x44	0x44
Byte at offset 1 (`$t1`)	0x43	0x43
Byte at offset 2 (`$t2`)	0x42	0x42
Byte at offset 3 (`$t3`)	0x41	0x41
Whole word (`$t4`)	ABCD	ABCD

Question 3.2 (Analyze) Which byte of `0x41424344` is at the lowest address in MARS, and which byte order is that? Does it match your prediction? Why does `$t4` still hold `0x41424344`, and why does a program that uses only `lw` and `sw` never notice the byte order?

0x44 is the byte with the lowest address (Little endian). In little endian, the byte order is reversed (by the standard of human-reading) for computing purpose, thus $t4 still hold 0x41424344 (because little endian doesnt actually reverse the bytes, it just change the reading order). processes that use only lw and sw never notice this detail because said processes never observe the order of byte to begin with.

Step 3.3 – Signed and unsigned bytes (6 points)

Listing 7 stores one byte of value `0xF0` and loads it twice. PREDICT `$t0` and `$t1` in hexadecimal and decimal, run it, and complete Table 14.

Listing 7

```
# Listing 7 - lb and lbu
.data
b1: .byte 0xF0
```

```
.text
.globl main
main:
    la   $s0, b1
    lb   $t0, 0($s0)
    lbu  $t1, 0($s0)
    li   $v0, 10
    syscall
```

Table 14. lb and lbu compared

Instruction	PREDICT (hex)	PREDICT (dec)	OBSERVE (hex)	OBSERVE (dec)
`lb $t0, 0($s0)`	0xfffffff0	-16	0xfffffff0	-16
`lbu $t1, 0($s0)`	0x0000000f0	240	0x0000000f0	240

Question 3.3 (Analyze) Explain the difference in terms of sign extension (compare LAB 1, Question 2.4). Which instruction would you use to read the brightness of a pixel stored as a number from 0 to 255, and why?

"lb" extend the left-most bit of the byte to fill the register. Thus, to read the brightness of a pixel stored as a number from 0 to 255, it is much safer to use "lbu", as the variable to store the brightness can be stored as a word, by which a sign extension by "lbu" may cause unexpected error in further calculations.

Step 3.4 – Capital letters by hand (7 points)

A capital letter has a code 32 below the matching lower-case letter (Table 18). Complete Listing 8 so that it changes the letters at offsets 0, 7 and 11 of `str1` into capital letters: for each one, load the byte with `lb`, subtract 32 and store it back with `sb`. The program must print `Nguyen Van An`. Then change `.asciiz` to `.ascii` in the declaration of `str1`, PREDICT the output, run it and OBSERVE (Table 15). Restore `.asciiz` afterwards.

Listing 8

```
# Listing 8 - capital letters at chosen positions of a string
# Register plan:  $s0 = address of the first byte of str1
.data
str1: .asciiz "nguyen van an"
tag:  .asciiz " [end]"
nl:   .asciiz "\n"

.text
.globl main
main:
    la   $s0, str1        # $s0 = address of str1
    # >>> make the byte at offset 0 a capital letter
    # >>> make the byte at offset 7 a capital letter
    # >>> make the byte at offset 11 a capital letter
    li   $v0, 4           # service 4: print the string that starts at $a0
    add  $a0, $s0, $zero
    syscall
    li   $v0, 4
    la   $a0, nl
    syscall
    li   $v0, 10
    syscall
```

Screenshot of the Run I/O pane (with `.asciiz`):

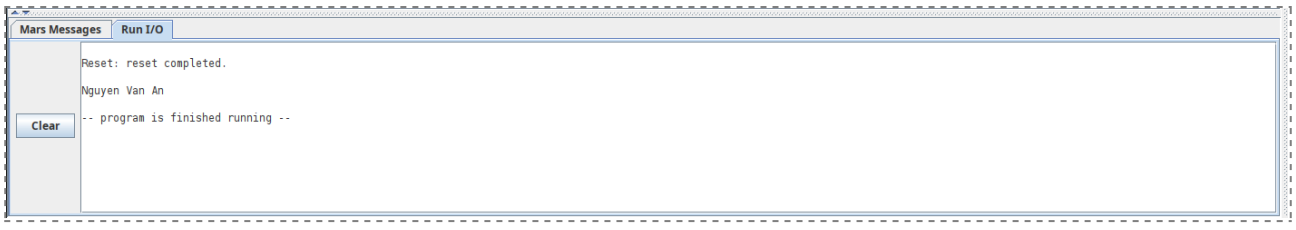


Table 15. A string without the terminating zero

Change	PREDICT	OBSERVE
`.ascii` instead of `.asciiz`	Nguyen Van An [end]	Nguyen Van An [end]

Question 3.4 (Analyze) Explain the output you observed with `.ascii`. Where did the extra text come from, and how does the print service know where a string ends?

The extra text come from the next defined string after the ".ascii". This additional text appears because our string is not terminated by NULL. Because the computer only know that the string ends if it is NULL-terminated, the computer will treat any non-NULL byte attached after the string part of it.

EXTENSION. A 3 × 3 BOARD IN MEMORY (optional – up to 10 bonus points)

A game board with 3 rows and 3 columns is stored in memory as nine words, row by row: `board: .word 0, 0, 0, 0, 0, 0, 0, 0, 0`. The cell in row `r` and column `c` (both counted from 0) is word number 3 × r + c, so its address is `board` + 4 × (3 × r + c). Write a program that keeps `r` in `$s1` and `c` in `$s2`, computes this address with `sll` and `add` (3 × r is r + 2 × r), and stores the value 1 (mark X) into the cell (1, 2) and the value 2 (mark O) into the cell (2, 0). After the run, the Data Segment must show 1 and 2 in two of the nine words and 0 in all the others.

Question E.1 (Analyze) Which two words of `board` changed, at which addresses, and why does the formula give these words?

the 6th and 7th words of the board changed (counting from 1). Addresses are 0x10010014 and 0x 10010018 respectively.
the calculation gives 1*3+2=5 and 2*3+0=6, which is the 6th and 7th words.

PART 4. WRAP-UP AND SUBMISSION (15 points)

Question 4.1 (Understand) Explain why `varA = varB + varC` needs two loads and one store in addition to the `add` when the variables are in memory. Use the terms *register*, *main memory*, *load* and *store*. (5 points)

because the process can only perform operations between registers, and not directly on the main memory, we must use 2 load to load varB and varC for calculation, and then one store to save the result to varA.

Question 4.2 (Apply) Write the MIPS instructions for `scores[3] = scores[1] + scores[2]`. Assume `$s0` holds the address of `scores[0]` and use only `$t` registers. (5 points)

```
.data
scores: .word 40, 55, 70, 85, 100
nl:     .asciiz "\n"

.text
.globl main
main:
    la   $s0, scores
    lw   $t0, 4($s0)
    lw   $t1, 8($s0)
    add  $t2, $t0, $t1
    sw   $t2, 12($s0)
     lw   $a0, 12($s0)
     li   $v0, 1
    syscall

    li   $v0, 4
    la   $a0, nl
    syscall
```

```
li   $v0, 10
syscall
```

Question 4.3 (Evaluate) `lw` and `sw` need an address that is a multiple of 4, but `lb` and `sb` accept any address. Explain why, and name one task from this LAB where a byte instruction is the right choice and one where a word instruction is. (5 points)

because lw and sw work on words, which is a data type that occupy 4 bytes, thus it need an address that is multiple of 4; while lb and sb work on bytes, which obviously occupy one byte, thus need no requirement for its address.
Task 3.4 requires the student to manipulate 3 bytes in the string, thus we must implement a code using sb and lb instead of lw and sw.
Task 2.3 requires the student to save a value to an array of words, and calculate the sum, thus we must implement a code using lw and sw to calculate and save values instead of using sb and lb.

Table 16. Submission checklist

☐	Item
x	The files `Lab02_<StudentID>_TN1.asm` to `_TN3.asm` (and `_EXT.asm` if you did the extension) are saved with the correct names, begin with a comment block and assemble without errors.
x	Every `# >>>` marker is replaced by your own code and the register plan is documented in comments.
x	This document is complete: all yellow cells are filled and every dashed frame contains a screenshot.
x	The `.asm` files are pushed to your GitHub repository, the link is written on the first page, and everything is submitted as announced by your instructor.

APPENDIX. REFERENCE

Table 17. Instructions and system calls used in this LAB

Instruction	Example	Meaning
`lw / sw`	`lw $t0, 8($s0)`	`$t0` = word at address `$s0 + 8` / word at that address = `$t0` (address a multiple of 4)
`lb / lbu`	`lb $t0, 1($s0)`	`$t0` = byte at address `$s0 + 1`, sign-extended / filled with zeros
`sb`	`sb $t0, 1($s0)`	byte at address `$s0 + 1` = lowest 8 bits of `$t0`
`la, li`	`la $s0, label`	pseudo-instructions: `$s0` = address of `label`; `li $t0, 7`: `$t0` = 7
`add, addi`	`addi $t0, $t1, 20`	`$t0` = `$t1` + 20 (`add` uses two registers)
`sll`	`sll $t1, $s1, 2`	`$t1` = `$s1` shifted left by 2 bits (4 × `$s1`)
`syscall`	`$v0 = 1, 4, 10`	print integer (`$a0`); print string (`$a0` = address); exit

Table 18. ASCII codes used in this LAB

Character	Decimal	Hexadecimal	Remark
end of string	0	`0x00`	terminator of `.asciiz`
space; `!`	32; 33	`0x20; 0x21`	
digits `0` to `9`	48 – 57	`0x30 - 0x39`	consecutive
capital letters `A` to `Z`	65 – 90	`0x41 - 0x5A`	consecutive
lower-case letters `a` to `z`	97 – 122	`0x61 - 0x7A`	consecutive